

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas
Informáticos



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Máster
Deep Learning
Universidad Politécnica de Madrid

Trabajo Fin de Máster

Sistema multiagente para análisis financiero histórico
mediante generación automática de código

MÁSTER DE FORMACIÓN PERMANENTE EN DEEP LEARNING

Presentado por:

Carlos Ruiz Oyarzun

Bajo la dirección de:

Dr. Adrián Girón Jiménez

Dr. Alejandro Delgado Peribáñez

Madrid, 2026

Resumen

Este Trabajo Fin de Máster propone el desarrollo incremental de un sistema multiagente orientado al análisis financiero histórico. El sistema parte de consultas ya normalizadas, datos de mercado almacenados en CSV y un conjunto cerrado de intenciones analíticas. A partir de esa entrada, el prototipo valida la solicitud, selecciona una familia de agente, genera un plan de análisis, construye código Python ejecutable, lanza dicho código en un proceso controlado y transforma la salida en una respuesta interpretable. La arquitectura se ha construido de forma gradual, manteniendo una base determinista y trazable antes de incorporar de forma controlada modelos de lenguaje mediante APIs compatibles con OpenAI.

El MVP actual soporta seis intenciones analíticas: crecimiento de precio, comparación de activos, visión descriptiva de un activo, análisis de retornos, análisis de riesgo histórico y análisis técnico. La evaluación combina pruebas funcionales, escenarios de error controlado y una comparativa entre el modo determinista y varios proveedores LLM. Los resultados muestran que el sistema puede mantener el cálculo financiero bajo control programático y utilizar modelos de lenguaje como una capa de interpretación final para mejorar la redacción sin alterar las métricas calculadas.

Palabras clave: sistema multiagente, análisis financiero, modelos de lenguaje, generación de código, reproducibilidad, fallback determinista.

Abstract

This Master's Thesis proposes the incremental development of a multi-agent system for historical financial analysis. The system starts from already normalized queries, market data stored in CSV files and a closed set of analytical intents. From this input, the prototype validates the request, selects an agent family, generates an analysis plan, builds executable Python code, runs that code in a controlled process and transforms the output into an interpretable response. The architecture has been built gradually, preserving a deterministic and traceable base before incorporating optional calls to language models through APIs compatible with OpenAI.

The current MVP supports six analytical intents: price growth, asset comparison, descriptive view of an asset, returns analysis, historical risk analysis and technical analysis. The evaluation combines functional tests, controlled error scenarios and a comparison between the deterministic mode and several LLM providers. The results show that the system can keep financial computation under programmatic control while using language models only as an optional layer to improve the final wording of the response.

Keywords: multi-agent system, financial analysis, language models, code generation, reproducibility, deterministic fallback.

Índice general

Resumen		I
Abstract		II
1. Introduccion		1
1.1. Contexto y definicion del problema		1
1.2. Motivacion		1
1.3. Objetivos		2
1.4. Alcance		2
1.5. Estructura de la memoria		2
2. Estado de la cuestion		4
2.1. Analisis financiero historico asistido por software		4
2.2. Sistemas basados en modelos de lenguaje		4
2.3. Arquitecturas multiagente		6
2.4. Limitaciones de los enfoques monoliticos		6
2.5. Generacion automatica de codigo		6
2.6. Herramientas y tecnologias relacionadas		7
2.7. Justificacion de la propuesta		7
3. Datos y material de trabajo		8
3.1. Fuente de datos y trazabilidad		8
3.2. Catalogo de consultas y activos		8
3.3. Estructura de los CSV		9
3.4. Analisis exploratorio de datos		9
3.5. Calidad, cobertura y limitaciones		10
3.6. Papel de los notebooks auxiliares		10
4. Metodologia		11
4.1. Enfoque incremental del desarrollo		11
4.2. Definicion de intenciones analiticas		12
4.3. Diseno experimental		12

4.4.	Metricas de validacion	12
4.5.	Justificacion de no entrenar un modelo propio	13
4.6.	Gestion de riesgos tecnicos	13
5.	Diseno del sistema	14
5.1.	Arquitectura general	14
5.2.	Entrada estructurada y estado compartido	14
5.3.	Nodos del pipeline	14
5.4.	Familias de agentes analiticos	15
5.5.	Generacion controlada de codigo	15
5.6.	Ejecucion segura y artefactos	15
5.7.	Gestion de errores y fallback determinista	15
6.	Implementacion	17
6.1.	Estructura del repositorio	17
6.2.	Modulos principales	17
6.3.	Ejecucion, despliegue local y reproducibilidad	18
6.4.	Tests automatizados	18
6.5.	Evaluacion automatica de perfiles LLM	18
6.6.	Artefactos generados	19
7.	Integracion controlada de modelos de lenguaje	20
7.1.	Motivacion de la capa LLM	20
7.2.	Cliente compatible con OpenAI	20
7.3.	Activacion granular	20
7.4.	Proveedores evaluados	21
7.5.	Estrategia de fallback	21
7.6.	Observaciones tecnicas de la integracion	21
7.7.	Gestion de credenciales	22
8.	Resultados y evaluacion	23
8.1.	Validacion del modo determinista	23
8.2.	Robustez ante errores	23
8.3.	Comparativa entre proveedores LLM	24
8.4.	Evaluacion del modelo universitario vLLM	24
8.5.	Primera ejecucion comparativa completa	25
8.6.	Discusion de resultados	25
9.	Aspectos eticos, seguridad y uso responsable	26
9.1.	Delimitacion financiera del sistema	26
9.2.	Riesgos de interpretacion	26

9.3. Seguridad en la generacion y ejecucion de codigo	26
9.4. Privacidad y datos	27
9.5. Sostenibilidad, costes y recursos	27
9.6. Limitaciones de uso	27
10.Conclusiones y trabajo futuro	28
10.1. Cumplimiento de objetivos	28
10.2. Aportaciones principales	28
10.3. Limitaciones actuales	29
10.4. Lineas de trabajo futuro	29
Referencias	30
A. Anexos	31
A.1. Ejecucion del MVP	31
A.2. Configuracion de APIs	31
A.3. Documentacion interna de apoyo	32

Índice de tablas

3.1. Tipos de activos representados en el conjunto de datos.	9
3.2. Resumen de cobertura y calidad del conjunto de datos.	9
3.3. Metricas exploratorias por serie utilizadas para contextualizar los casos de prueba.	10
4.1. Evolucion incremental del MVP.	11
4.2. Intenciones analiticas soportadas por el MVP.	12
6.1. Principales modulos del repositorio.	17
8.1. Casos funcionales representativos del MVP.	23
8.2. Comparativa general de configuraciones evaluadas.	24
8.3. Resultados del perfil universitario vLLM.	24
8.4. Tiempos aproximados observados en la primera comparativa completa.	25

Índice de figuras

2.1. Mapa conceptual de inteligencia artificial, aprendizaje automatico, redes neuronales, aprendizaje profundo e inteligencia artificial generativa. Fuente: imagen divulgativa publicada por la cuenta de Instagram @analytics_vidhya.	5
--	---

Capítulo 1



Introduccion

1.1. Contexto y definicion del problema

El analisis financiero historico suele combinar varias tareas que, aunque son conceptualmente distintas, aparecen juntas en el trabajo practico: carga de datos, validacion de entradas, seleccion del tipo de analisis, calculo de metricas, generacion de graficos, interpretacion de resultados y comunicacion final al usuario. En muchos prototipos basados en modelos de lenguaje, estas fases se mezclan dentro de una unica llamada generativa, lo que dificulta la trazabilidad del resultado y aumenta el riesgo de errores silenciosos.

Este trabajo plantea una aproximacion diferente. En lugar de delegar todo el proceso en un modelo generativo, se disena un flujo multiagente en el que cada fase tiene una responsabilidad delimitada. El sistema no intenta resolver desde cero el problema completo de comprension del lenguaje natural, sino que parte de una consulta financiera ya estructurada: intencion, tickers, rango temporal, intervalo y rutas a los ficheros CSV. A partir de esta entrada, el objetivo es orquestar un analisis reproducible y controlado.

1.2. Motivacion

La motivacion principal del proyecto es explorar como una arquitectura multiagente puede aportar orden, modularidad y robustez a un sistema de analisis financiero asistido por inteligencia artificial. La aplicacion de modelos de lenguaje al dominio financiero resulta atractiva por su capacidad de generar explicaciones y adaptar el lenguaje al usuario, pero tambien exige mecanismos de control. En un contexto financiero, incluso cuando el objetivo es puramente descriptivo, resulta esencial separar los calculos verificables de la redaccion final.

Por esta razon, el MVP se ha construido inicialmente con plantillas deterministas. Esta decision permite validar el flujo completo antes de introducir APIs externas o

modelos servidos localmente. Una vez estabilizada la arquitectura, la incorporacion de proveedores como Groq, Gemini o el **endpoint universitario vLLM** se realiza de forma gradual, empezando por la fase de interpretacion y conservando fallbacks programaticos.

1.3. Objetivos

El objetivo general del trabajo es desarrollar y validar un prototipo multiagente capaz de ejecutar analisis financieros historicos a partir de entradas estructuradas y datos en CSV.

Los objetivos especificos son:

- Definir un workflow modular que separe ingesta, enrutado, planificacion, generacion de codigo, ejecucion e interpretacion.
- Implementar familias de agentes especializadas para distintas intenciones analiticas.
- Mantener una base determinista que permita pruebas reproducibles y control de errores.
- Preparar la arquitectura para integrar modelos de lenguaje mediante APIs compatibles con OpenAI.
- Evaluar el MVP mediante casos funcionales y escenarios de error controlado.

1.4. Alcance

El sistema se limita al analisis historico-descriptivo de activos financieros a partir de datos ya descargados. No realiza prediccion futura, recomendacion de inversion ni analisis fundamental complejo. Esta restriccion es deliberada: el foco del TFM no es construir un asesor financiero, sino estudiar la arquitectura de un pipeline multiagente trazable para analisis historico.

1.5. Estructura de la memoria

La memoria se organiza siguiendo la logica de desarrollo del proyecto. En primer lugar se presenta el estado de la cuestion, con especial atencion a modelos de lenguaje, arquitecturas multiagente y limitaciones de enfoques monoliticos. A continuacion se describen los datos utilizados, su trazabilidad y su calidad. Despues se expone la metodologia incremental seguida durante el desarrollo.

Los capitulos centrales describen el diseno del sistema, la implementacion del MVP y la integracion controlada de modelos de lenguaje. Posteriormente se presentan los resultados de evaluacion, incluyendo el modo determinista y la comparativa con proveedores LLM. Finalmente se discuten los aspectos eticos y de seguridad, se resumen las conclusiones y se plantean lineas de trabajo futuro.

Capítulo 2

Estado de la cuestion

2.1. Analisis financiero historico asistido por software

El analisis financiero historico se apoya habitualmente en el procesamiento de series temporales de precios. Las tareas mas frecuentes incluyen calculo de rentabilidades, comparacion de activos, estimacion de volatilidad, analisis de drawdown e indicadores tecnicos. Estas operaciones son reproducibles cuando se definen con claridad los datos de entrada, el periodo temporal y las formulas empleadas.

En este contexto, una herramienta automatizada debe priorizar la trazabilidad. No basta con producir una respuesta final correcta en apariencia: tambien es necesario poder revisar que datos se han utilizado, que transformaciones se han aplicado y que metricas se han calculado.

2.2. Sistemas basados en modelos de lenguaje

Los modelos de lenguaje de gran escala han abierto la posibilidad de construir sistemas capaces de interpretar instrucciones, generar codigo y redactar explicaciones adaptadas al usuario. Sin embargo, su uso directo en tareas analiticas puede introducir problemas de verificabilidad si no se acompaña de estructuras de control. En dominios donde los resultados dependen de calculos concretos, como el analisis financiero, resulta conveniente combinar componentes generativos con ejecucion programatica y validaciones explicitas.

La Figura 2.1 permite situar visualmente el alcance tecnologico del trabajo.

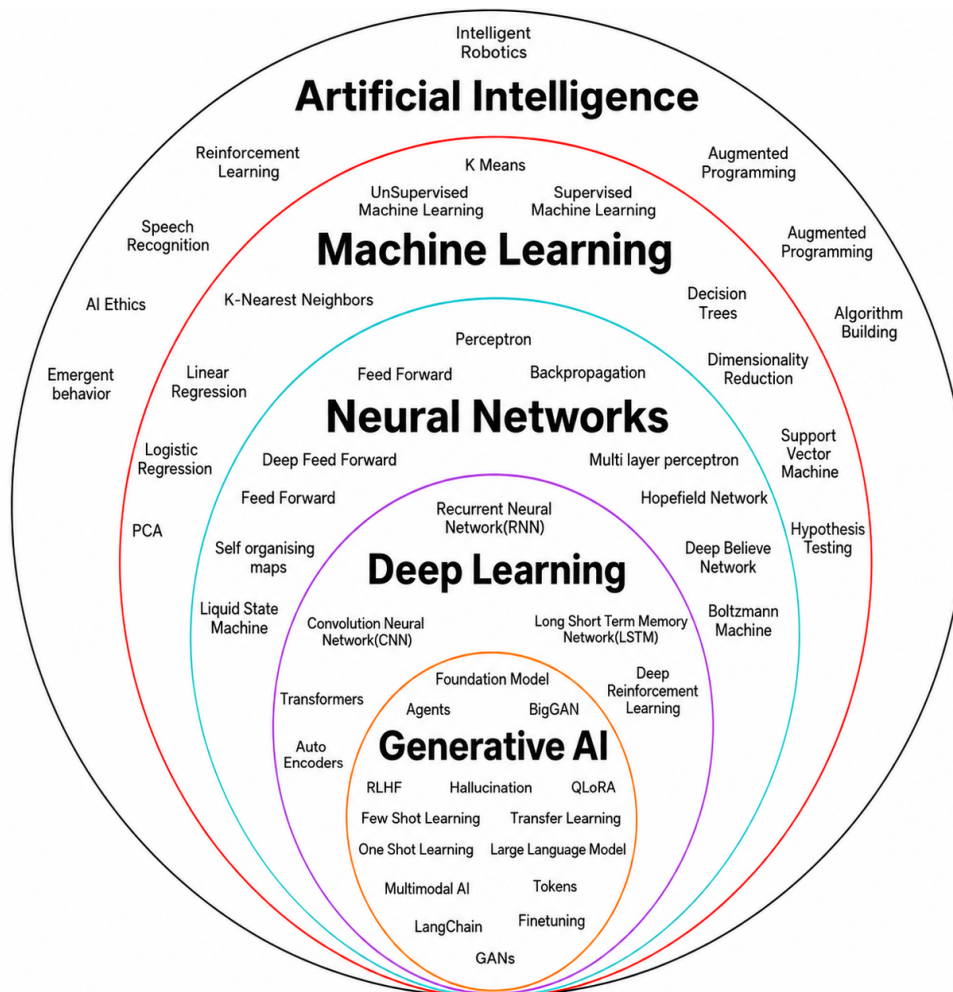


Figura 2.1: Mapa conceptual de inteligencia artificial, aprendizaje automatico, redes neuronales, aprendizaje profundo e inteligencia artificial generativa. Fuente: imagen divulgativa publicada por la cuenta de Instagram @analytics_vidhya.

Aunque se trata de una imagen divulgativa y no de una taxonomía formal exhaustiva, resulta útil para contextualizar el lugar que ocupan los modelos de lenguaje dentro del campo de la inteligencia artificial. La inteligencia artificial constituye el marco más amplio, orientado al desarrollo de sistemas capaces de realizar tareas que requieren algún grado de comportamiento inteligente. Dentro de ella, el aprendizaje automático agrupa métodos que aprenden patrones a partir de datos, mientras que las redes neuronales representan una familia concreta de modelos parametrizados. El aprendizaje profundo se apoya en redes neuronales con múltiples capas, capaces de aprender representaciones jerárquicas de la información. Finalmente, la inteligencia artificial generativa aparece como un subconjunto asociado a modelos capaces de producir contenido nuevo, como texto, código, imágenes o respuestas multimodales.

Esta distinción es importante para el presente TFM porque el sistema desarrollado no aborda el entrenamiento de modelos de aprendizaje profundo ni sustituye el cálculo financiero por inferencia generativa. El uso de modelos de lenguaje se plantea como

una capa auxiliar y acotada sobre un pipeline financiero trazable. En la configuracion actual, el LLM se limita principalmente a la interpretacion textual de resultados ya calculados por codigo determinista. De este modo, la arquitectura combina una base verificable, centrada en datos historicos y metricas reproducibles, con una capa generativa orientada a mejorar la claridad de la respuesta final sin asumir la responsabilidad del calculo financiero.

2.3. Arquitecturas multiagente

Las arquitecturas multiagente permiten dividir una tarea compleja en responsabilidades mas pequenas. En este proyecto, la idea de agente no se entiende como una entidad autonoma sin restricciones, sino como una familia analitica con una interfaz clara: recibe una intencion, produce un plan, genera o selecciona codigo y contribuye a la interpretacion final. Esta separacion facilita la extension del sistema, ya que nuevas intenciones pueden incorporarse sin reescribir el flujo completo.

2.4. Limitaciones de los enfoques monoliticos

Un enfoque monolitico, en el que una unica funcion o prompt realiza todo el proceso, puede ser suficiente para una demostracion pequena, pero presenta limitaciones cuando se desea evaluar el sistema de forma rigurosa. Entre ellas destacan la dificultad para aislar errores, la falta de trazabilidad entre la entrada y la salida, y la ausencia de puntos intermedios donde aplicar validaciones. El diseno propuesto en este TFM busca precisamente evitar estas limitaciones.

2.5. Generacion automatica de codigo

Una de las capacidades mas relevantes de los modelos de lenguaje es la generacion de codigo. En un sistema de analisis financiero, esta capacidad puede resultar util para adaptar el calculo a distintas intenciones. No obstante, tambien introduce riesgos: errores sintacticos, calculos incorrectos, uso de columnas inexistentes o dependencias no controladas.

Por esta razon, el presente trabajo diferencia entre generacion controlada de codigo y generacion abierta mediante LLM. En la version actual del MVP, el codigo se genera mediante plantillas programaticas para garantizar estabilidad. La posible generacion mediante modelo se plantea como una extension futura y no como requisito para validar la arquitectura base.

2.6. Herramientas y tecnologías relacionadas

El proyecto se apoya en tecnologías ampliamente utilizadas en sistemas de análisis y automatización. Python se utiliza como lenguaje principal por su ecosistema de procesamiento de datos. Los datos financieros proceden de `yfinance`. La orquestación se diseña siguiendo una lógica de workflow con estado compartido, compatible con herramientas como LangGraph. La integración LLM se prepara mediante una interfaz compatible con OpenAI, lo que permite probar Groq, Gemini y un endpoint universitario basado en vLLM.

2.7. Justificación de la propuesta

La propuesta del TFM se justifica por la necesidad de combinar dos propiedades que a menudo aparecen separadas: cálculo trazable y explicación flexible. El sistema mantiene los cálculos financieros en código controlado, pero permite incorporar modelos de lenguaje en fases concretas, especialmente en la interpretación final. De este modo se reduce el riesgo de alucinaciones en métricas y se conserva la posibilidad de generar respuestas más naturales.

Capítulo 3

Datos y material de trabajo

3.1. Fuente de datos y trazabilidad

El sistema trabaja con datos financieros historicos descargados previamente mediante `yfinance` y almacenados en ficheros CSV dentro del repositorio. Cada fichero de datos dispone de un fichero de metadatos asociado en formato JSON, donde se conserva informacion sobre la consulta original, la intencion analitica, los tickers implicados, el intervalo temporal y los parametros utilizados para la descarga.

Esta decision permite separar la fase de obtencion de datos de la fase de analisis. El MVP no depende de que la API externa este disponible durante cada ejecucion, sino que opera sobre un conjunto de datos congelado y versionado. De este modo, los resultados obtenidos son mas reproducibles y pueden revisarse posteriormente a partir de los mismos ficheros de entrada.

3.2. Catalogo de consultas y activos

El catalogo de casos se encuentra en `data/catalog/query_cases.json`. En la version actual se han definido diez consultas de referencia y se han analizado ocho CSV, que contienen diez series ticker-caso normalizadas. Los activos incluidos cubren acciones, ETFs, un indice amplio, un criptoactivo, una divisa y un futuro de materia prima.

La variedad de activos permite comprobar que el sistema no esta limitado a un unico instrumento financiero y que puede trabajar con diferentes horizontes temporales e intervalos de observacion.

Categoría	Activos incluidos
Acciones	AAPL, AMD, NVDA
ETFs	QQQ, SPY
Indice	S&P 500 (^GSPC)
Criptoactivo	BTC-USD
Divisa	EURUSD=X
Materia prima / futuro	GC=F

Tabla 3.1: Tipos de activos representados en el conjunto de datos.

3.3. Estructura de los CSV

Los ficheros generados por `yfinance` contienen columnas de tipo OHLCV: apertura, maximo, minimo, cierre, cierre ajustado y volumen. En algunos casos se almacenan con cabeceras multinivel, por lo que el proyecto incorpora una capa de lectura y normalizacion antes de ejecutar los calculos financieros.

La variable mas importante para el MVP es el precio de cierre, ya que sobre ella se calculan crecimiento, rentabilidades, volatilidad, drawdown e indicadores tecnicos basicos. El volumen se conserva como informacion complementaria, aunque no se trata como obligatorio para todos los activos, especialmente en divisas o futuros intradia.

3.4. Analisis exploratorio de datos

El analisis exploratorio confirma que el conjunto de datos contiene 5.080 filas normalizadas en formato largo, con un rango temporal observado entre el 2 de enero de 2020 y el 17 de marzo de 2026. Los intervalos considerados son diario (1d) e intradia horario (1h).

Indicador	Resultado
Casos definidos en catalogo	10
CSV analizados	8
Series ticker-caso	10
Filas normalizadas	5.080
Intervalos	1d, 1h
Nulos en precios de cierre	0
Duplicados por fecha	0 en los ficheros analizados

Tabla 3.2: Resumen de cobertura y calidad del conjunto de datos.

Ademas del resumen agregado, se calcularon metricas descriptivas por serie para comprobar que los casos de prueba cubrian situaciones financieras diversas. La Tabla 3.3 muestra una seleccion de las series analizadas. Se observan activos con crecimientos muy

elevados, como NVDA en cinco años, series con drawdowns significativos, como AMD en la comparativa con NVDA, y activos de menor volatilidad relativa, como SPY. Esta diversidad es útil porque obliga al sistema a redactar respuestas para escenarios con magnitudes y niveles de riesgo distintos.

Serie	Retorno total	Volatilidad anual	Max. drawdown
NVDA, 5 años	1273.33 %	51.31 %	-66.36 %
BTC-USD, 2024	109.76 %	44.13 %	-26.18 %
NVDA, 2 años	107.13 %	49.42 %	-36.89 %
S&P 500, desde 2020	105.64 %	20.77 %	-33.93 %
QQQ, 2024	28.07 %	17.99 %	-13.56 %
SPY, 2024	24.45 %	12.63 %	-8.41 %
AMD, 2 años	3.11 %	55.10 %	-58.98 %
AAPL, 3 meses	-7.00 %	24.15 %	-10.07 %

Tabla 3.3: Métricas exploratorias por serie utilizadas para contextualizar los casos de prueba.

3.5. Calidad, cobertura y limitaciones

La ausencia de nulos en el precio de cierre es relevante porque las métricas principales del MVP dependen directamente de esta columna. También se comprobó la ausencia de duplicados por fecha en los ficheros analizados. Estas condiciones permiten ejecutar las seis intenciones actuales sin introducir imputaciones adicionales.

Como limitación, las descargas basadas en periodos relativos, por ejemplo 3mo o 5y, dependen de la fecha en la que se genere el CSV. Para una versión final completamente cerrada sería recomendable fijar rangos `start/end` en todos los casos. También debe considerarse que los datos proceden de Yahoo Finance mediante `yfinance`, por lo que en un entorno productivo habría que estudiar licencias, disponibilidad y estabilidad del proveedor.

3.6. Papel de los notebooks auxiliares

Durante el desarrollo se conservaron tres notebooks con funciones diferenciadas. El notebook `01_yfinance_eda_avanzado.ipynb` se mantiene como laboratorio para probar nuevos tickers, periodos e intervalos antes de incorporarlos al dataset definitivo. El notebook `02_catalogo_queries_y_csv.ipynb` documenta la relación entre el catálogo de consultas y los ficheros CSV esperados. Finalmente, `03_eda_datos_tfm.ipynb` queda como referencia principal del análisis exploratorio formal, junto con las tablas procesadas en `data/processed/`.

Capítulo 4

Metodologia

4.1. Enfoque incremental del desarrollo

El desarrollo del trabajo se ha planteado como una sucesion de iteraciones controladas. En lugar de construir desde el inicio un sistema completamente dependiente de modelos de lenguaje, se ha partido de un MVP determinista capaz de completar el flujo de extremo a extremo. Esta decision permite validar primero la arquitectura, la trazabilidad y la ejecucion del codigo generado antes de introducir componentes generativos mas variables.

La primera iteracion valido el flujo minimo con dos intenciones analiticas: crecimiento historico y comparacion de activos. La segunda iteracion amplio el alcance con analisis descriptivo y analisis de retornos. La tercera incorpore analisis de riesgo historico, analisis tecnico, pruebas de error y una reorganizacion interna basada en familias de agentes. La Tabla 4.1 resume la progresion seguida.

Iteracion	Objetivo principal	Evidencia obtenida
MVP 1	Validar el flujo extremo a extremo con <code>price_growth</code> y <code>compare_assets</code> .	El sistema procesa un caso de activo unico y un caso multi-activo, genera codigo ejecutable y produce una respuesta interpretable.
MVP 2	Comprobar que la arquitectura crece sin rediseñarse, incorporando <code>asset_overview</code> y <code>return_analysis</code> .	Se pasa de dos a cuatro intenciones y se valida que las nuevas metricas se integran sobre las mismas interfaces.
MVP 3	Aumentar madurez funcional y robustez mediante <code>historical_risk_analysis</code> , <code>technical_analysis</code> y pruebas negativas.	Se consolidan seis intenciones, tres escenarios de error controlado y una modularizacion por familias de agentes.

Tabla 4.1: Evolucion incremental del MVP.

4.2. Definicion de intenciones analiticas

El MVP trabaja con un conjunto cerrado de intenciones. Esta restriccion hace que el problema sea evaluable y reduce la ambigüedad del sistema. Cada intencion se asocia con una familia de agente, un plan analitico, un bloque de codigo y una forma de interpretar la salida.

Intencion	Objetivo
<code>price_growth</code>	Calcular crecimiento o caida de un activo en un periodo.
<code>compare_assets</code>	Comparar la evolucion relativa de varios activos.
<code>asset_overview</code>	Generar una vision descriptiva general de un activo.
<code>return_analysis</code>	Analizar retornos historicos y estadisticos basicos.
<code>historical_risk_analysis</code>	Estimar riesgo historico mediante volatilidad y draw-down.
<code>technical_analysis</code>	Calcular indicadores tecnicos como medias moviles y RSI.

Tabla 4.2: Intenciones analiticas soportadas por el MVP.

4.3. Diseno experimental

La evaluacion se apoya en seis casos representativos, uno por cada intencion implementada. Estos casos permiten comprobar que el sistema valida la entrada, selecciona la familia analitica correcta, genera un plan, construye codigo ejecutable, ejecuta el script y devuelve una respuesta final interpretable.

Ademas de los casos correctos, se incorporan escenarios negativos para comprobar la robustez del flujo. Entre ellos se incluyen intenciones no soportadas, rutas CSV inexistentes y cardinalidad incorrecta de tickers para analisis que requieren una unica serie.

4.4. Metricas de validacion

La validacion del MVP no se plantea como entrenamiento de un modelo supervisado, sino como evaluacion funcional de un pipeline software. Por tanto, las metricas principales son:

- Exitos de ejecucion del flujo completo.
- Codigo de retorno del script generado.
- Presencia o ausencia de errores controlados.

- Cobertura de intenciones analíticas.
- Uso de fallback determinista cuando falla una API externa.
- Tiempo de respuesta en las pruebas con proveedores LLM.

4.5. Justificación de no entrenar un modelo propio

El objetivo del TFM no es entrenar desde cero un modelo financiero, sino estudiar una arquitectura multiagente trazable para automatizar análisis históricos. Por ello, el trabajo utiliza modelos de lenguaje preentrenados a través de APIs compatibles con OpenAI y limita su papel, en la configuración actual, a la fase de interpretación final.

Esta decisión reduce el coste computacional, evita depender de hardware GPU propio y permite centrar la evaluación en la robustez del workflow. La planificación, el cálculo financiero y la ejecución permanecen bajo control programático.

4.6. Gestión de riesgos técnicos

Los principales riesgos identificados son la fragilidad del código generado, la disponibilidad de APIs externas, los problemas de codificación en respuestas LLM, las cuotas de proveedores gratuitos y la posible confusión entre análisis histórico y recomendación financiera. Para mitigarlos, el sistema conserva una base determinista, valida entradas, captura salidas de ejecución, registra artefactos y activa la capa LLM de forma controlada, empezando por la interpretación final.

Capítulo 5

Diseno del sistema

5.1. Arquitectura general

El sistema se ha disenado como un workflow modular que transforma una consulta financiera estructurada en una respuesta analitica basada en calculos ejecutados sobre datos historicos. La arquitectura separa de forma explicita las fases de ingesta, enrutado, planificacion, generacion de codigo, ejecucion e interpretacion.

Esta separacion evita que una unica llamada generativa concentre todo el razonamiento del sistema. Cada fase tiene una responsabilidad concreta y produce artefactos intermedios que pueden revisarse, probarse y sustituirse de forma independiente.

5.2. Entrada estructurada y estado compartido

El flujo parte de una entrada normalizada que incluye consulta original, intencion analitica, tickers, periodo o fechas, intervalo y rutas a los CSV. A partir de esa entrada se construye un estado compartido que viaja por los nodos del workflow.

El estado contiene, entre otros campos, la intencion detectada, el agente seleccionado, el plan analitico, el codigo generado, la salida estandar, la salida de error, el codigo de retorno, los avisos y la respuesta final. Esta estructura permite mantener trazabilidad entre la entrada inicial y el resultado entregado al usuario.

5.3. Nodos del pipeline

El pipeline se organiza en seis nodos principales:

1. **Ingesta**: valida la entrada minima y comprueba la existencia de los ficheros.
2. **Router**: valida la coherencia de la solicitud y selecciona la familia analitica.
3. **Analisis especializado**: transforma la intencion en un plan estructurado.

4. **Generacion de codigo:** construye un script Python adaptado al plan.
5. **Ejecucion:** ejecuta el script en un proceso controlado.
6. **Interpretacion:** convierte los resultados en una respuesta legible.

5.4. Familias de agentes analiticos

La logica especifica de cada intencion se agrupa en familias de agentes. En esta version, la idea de agente no se interpreta como una entidad autonoma sin restricciones, sino como un componente especializado con un contrato claro. Cada agente sabe que metricas calcular, que salidas esperar y como construir una interpretacion inicial.

Esta organizacion facilita la extension del sistema. Si se desea incorporar una nueva intencion, se puede anadir una nueva familia analitica sin reescribir el workflow completo.

5.5. Generacion controlada de codigo

La generacion de codigo se situa entre el plan analitico y la ejecucion. En el modo determinista, el sistema utiliza plantillas programaticas que producen scripts Python ajustados al contrato esperado. Esta decision permite controlar imports, rutas, formato de salida y gestion de errores.

En futuras iteraciones, la generacion de codigo podria delegarse parcialmente en un modelo de lenguaje. Sin embargo, esta activacion se considera mas arriesgada que la interpretacion final, por lo que el MVP actual mantiene esta fase bajo control determinista.

5.6. Ejecucion segura y artefactos

El codigo se guarda como un script independiente y se ejecuta mediante un proceso externo controlado. La ejecucion captura `stdout`, `stderr`, `returncode` y un payload estructurado con los resultados del analisis.

Los scripts generados y los logs de ejecucion se almacenan como artefactos revisables. Esto permite auditar que calculos se realizaron y que salida se utilizo para construir la respuesta final.

5.7. Gestion de errores y fallback determinista

El sistema incorpora validaciones para errores previsibles, como intenciones no soportadas, CSV inexistentes o numero incorrecto de tickers. Cuando se activa la capa

LLM, el flujo conserva un fallback determinista. Si la API falla, no esta configurada o devuelve una respuesta no valida, el sistema puede continuar con la logica programatica.

Esta estrategia es clave para que el MVP no dependa criticamente de proveedores externos ni de disponibilidad de red.

Capítulo 6

Implementacion

6.1. Estructura del repositorio

El codigo del proyecto se organiza en una estructura separada por responsabilidades. La carpeta `src/` contiene la implementacion principal, `data/` conserva los datos y catalogos, `notebooks/` incluye el analisis exploratorio y `tests/` agrupa las pruebas automatizadas.

Ruta	Funcion
<code>src/schemas.py</code>	Define modelos de entrada, salida y estado.
<code>src/graph/</code>	Contiene nodos, routing y construccion del workflow.
<code>src/agents/</code>	Agrupar las familias analiticas del MVP.
<code>src/execution/</code>	Gestiona persistencia y ejecucion de scripts.
<code>src/io/</code>	Lee y normaliza los CSV financieros.
<code>src/llm/</code>	Prepara la integracion controlada con APIs LLM.
<code>tests/</code>	Valida casos funcionales y escenarios de error.

Tabla 6.1: Principales modulos del repositorio.

6.2. Modulos principales

El fichero `schemas.py` define los contratos de datos que atraviesan el sistema. Los modulos de `src/graph/` implementan los nodos del pipeline y las reglas de transicion. Las familias de agentes se concentran en `src/agents/`, donde cada intencion se asocia con una logica analitica concreta.

La ejecucion de codigo se encapsula en `src/execution/code_runner.py`. Este modulo guarda el script generado, lo lanza como proceso externo y devuelve salidas estructuradas. Esta separacion evita mezclar el calculo financiero con la orquestacion del workflow.

6.3. Ejecucion, despliegue local y reproducibilidad

El MVP se ejecuta de forma local mediante `run_mvp.py`. La memoria considera este despliegue local como suficiente para la fase academica del trabajo, ya que el objetivo principal es validar el pipeline y no publicar un servicio productivo.

```
python run_mvp.py --example growth_nvda
python run_mvp.py --example compare_nvda_amd
python run_mvp.py --example overview_aapl
python run_mvp.py --example returns_qqq_spy
python run_mvp.py --example risk_qqq_spy
python run_mvp.py --example technical_aapl
```

La reproducibilidad se apoya en tres elementos: datos CSV versionados, ejemplos de entrada controlados y pruebas automatizadas. Las claves de API se almacenan en un fichero local `.env`, excluido del control de versiones.

6.4. Tests automatizados

El proyecto incorpora pruebas unitarias para validar tanto el modo determinista como la configuracion LLM. La bateria principal comprueba las seis intenciones soportadas y varios errores previsibles.

```
python -m unittest tests/test_mvp.py
python -m unittest tests/test_llm_config.py tests/test_mvp.py
```

En las ejecuciones documentadas, el MVP supera nueve pruebas sobre nueve en la bateria funcional principal. Al incluir las pruebas de configuracion LLM, se documenta una ejecucion de catorce pruebas superadas, lo que confirma que la incorporacion de dependencias LLM no rompe el funcionamiento determinista.

6.5. Evaluacion automatica de perfiles LLM

Para comparar el modo determinista con los distintos proveedores LLM se incorporo el script `scripts/evaluate_llm_profiles.py`. Este script ejecuta una misma bateria de ejemplos contra varios perfiles: `deterministic`, `groq`, `gemini` y `university`. Su objetivo es evitar evaluaciones manuales aisladas y generar evidencias comparables entre configuraciones.

Cada ejecucion registra el perfil utilizado, el ejemplo, la intencion analitica, el estado final, el codigo de retorno del script generado, posibles avisos, la respuesta final, el agente seleccionado, el plan analitico, la salida estructurada de ejecucion y el

tiempo aproximado. Tambien se incorporan comprobaciones cualitativas simples, como detectar si se ha usado fallback o si la respuesta evita formulaciones propias de una recomendacion directa de inversion.

```
python scripts/evaluate_llm_profiles.py \  
  --profiles deterministic groq gemini university --examples all
```

Los resultados generados se consideran artefactos de evaluacion y no codigo fuente. Por ello, las salidas completas pueden almacenarse en una carpeta de resultados ignorada por el control de versiones, mientras que la memoria recoge los indicadores agregados mas relevantes.

6.6. Artefactos generados

Cada ejecucion puede producir scripts, logs y payloads de resultado. Estos artefactos permiten revisar el comportamiento del sistema despues de la ejecucion y aportan evidencia para la memoria. La existencia de scripts generados tambien facilita detectar errores en la fase de calculo, en lugar de confiar unicamente en la respuesta final.

Capítulo 7

Integración controlada de modelos de lenguaje

7.1. Motivación de la capa LLM

Una vez consolidado el MVP determinista, el trabajo incorpora una capa de modelos de lenguaje. El objetivo no es sustituir el cálculo financiero, sino mejorar la redacción final y estudiar la portabilidad del sistema entre distintos proveedores.

La configuración recomendada activa inicialmente el LLM solo para interpretación. De este modo, el modelo recibe resultados ya calculados y redacta una explicación más natural, pero no modifica la selección del agente, el plan analítico ni el código ejecutado.

7.2. Cliente compatible con OpenAI

La integración se ha preparado mediante un cliente compatible con APIs tipo OpenAI. Esta decisión permite alternar entre proveedores sin modificar el workflow principal. El sistema utiliza variables de entorno para seleccionar perfil, modelo, URL base y clave de API.

7.3. Activación granular

La capa LLM puede activarse por fases mediante variables de configuración:

- `LLM_USE_FOR_INTERPRETATION`: el modelo redacta la respuesta final.
- `LLM_USE_FOR_PLANNING`: el modelo puede ajustar el plan analítico.
- `LLM_USE_FOR_CODEGEN`: el modelo puede generar código Python.

La activación de planificación y generación de código se mantiene deshabilitada en la evaluación principal, porque son fases con mayor riesgo de introducir errores no trazables.

7.4. Proveedores evaluados

Se han preparado y evaluado tres perfiles principales:

- **Groq**: proveedor externo compatible con OpenAI, utilizado como primera API operativa.
- **Gemini**: proveedor secundario para estudiar portabilidad y calidad comparativa.
- **Modelo universitario vLLM**: endpoint académico compatible con OpenAI Chat Completions, usando el modelo `openai/gpt-oss-20b`.

7.5. Estrategia de fallback

El diseño conserva siempre el modo determinista como fallback. Si una API falla por red, cuota, indisponibilidad o error de formato, el sistema puede devolver una respuesta programática válida. Esta propiedad es esencial para evitar que el MVP quede bloqueado por dependencias externas.

Durante las pruebas se observaron errores de cuota y disponibilidad en Gemini, así como un problema inicial de red en el entorno de ejecución. También se comprobó que, si falta la dependencia `openai`, el sistema no queda inutilizado: registra el aviso correspondiente y mantiene una respuesta determinista. En todos estos casos, el enfoque con fallback permitió mantener el sistema operativo.

7.6. Observaciones técnicas de la integración

La primera integración real con APIs externas puso de manifiesto varias cuestiones prácticas. Groq completó la batería de seis casos sin warnings una vez instaladas las dependencias y habilitada la conectividad, por lo que se consideró el proveedor externo más cómodo para iterar inicialmente. Gemini también produjo respuestas alineadas con los datos, pero mostró limitaciones operativas del *free tier*: errores 429 por cuota y 503 por alta demanda temporal, resueltos mediante espera, reintento y fallback determinista.

El endpoint universitario vLLM resultó especialmente relevante para el TFM porque confirma que la arquitectura puede conectarse a un backend académico compatible con OpenAI Chat Completions sin cambiar el workflow principal. En la validación técnica

se observo que el servidor devuelve un campo adicional `reasoning`; el sistema conserva esta informacion en la respuesta cruda, pero utiliza solo el contenido final del mensaje para construir la respuesta al usuario. Tambien se incorporo una reparacion defensiva de codificacion para evitar mojibake en respuestas con tildes o simbolos especiales.

7.7. Gestion de credenciales

Las claves de Groq, Gemini y del endpoint universitario se gestionan mediante variables de entorno en el fichero local `.env`. Este archivo no se versiona y no debe compartirse. El repositorio incluye configuraciones de ejemplo, pero una ejecucion real con API solo es posible cuando el entorno local contiene una clave valida. Si la clave falta, es un marcador de ejemplo o el perfil no esta configurado, el sistema debe continuar en modo determinista o activar fallback.

Capítulo 8

Resultados y evaluacion

8.1. Validacion del modo determinista

El resultado principal del trabajo es un MVP capaz de completar el flujo de analisis para seis intenciones financieras sin depender de APIs externas. En modo determinista, los seis casos representativos finalizan correctamente, sin warnings y con codigo de retorno cero en los scripts generados.

Caso	Intencion	Objetivo
growth_nvda	price_growth	Crecimiento historico de NVDA.
compare_nvda_amd	compare_assets	Comparacion de NVDA y AMD.
overview_aapl	asset_overview	Vision descriptiva de AAPL.
returns_qqq_spy	return_analysis	Analisis de retornos de QQQ y SPY.
risk_qqq_spy	historical_risk_analysis	Riesgo historico de QQQ y SPY.
technical_aapl	technical_analysis	Indicadores tecnicos de AAPL.

Tabla 8.1: Casos funcionales representativos del MVP.

8.2. Robustez ante errores

Ademas de los casos correctos, el MVP contempla errores de entrada. La bateria de pruebas incluye una intencion no soportada, un CSV inexistente y una cardinalidad incorrecta de tickers para `technical_analysis`. Estos escenarios se resuelven mediante estados de error controlados y respuestas explicativas.

Este comportamiento es relevante porque un sistema analítico no debe evaluarse solo en condiciones ideales. La capacidad de fallar de forma explicable forma parte de la calidad del prototipo.

8.3. Comparativa entre proveedores LLM

La evaluación compara cuatro configuraciones: modo **determinista**, Groq, Gemini y el modelo universitario vLLM. En todos los casos, el cálculo financiero permanece controlado por el pipeline del proyecto. Los LLM se utilizan solo para redactar la respuesta final.

Configuración	Casos OK	Fallbacks	Warnings	Lectura
Determinista	6/6	No aplica	0	Base estable y reproducible.
Groq	6/6	0	0	Mejor proveedor inicial.
Gemini	6/6	Si ante Cuotas 429/503		Util para comparación.
Universidad vLLM	6/6	0	0	Backend académico validado.

Tabla 8.2: Comparativa general de configuraciones evaluadas.

8.4. Evaluación del modelo universitario vLLM

El perfil universitario se configuró mediante una API compatible con OpenAI Chat Completions. La validación se realizó con `LLM_USE_FOR_INTERPRETATION=true` y con planificación y generación de código desactivadas.

Caso	Estado	Modo	Tiempo
<code>compare_nvda_amd</code>	completed	LLM directo	8.24 s
<code>growth_nvda</code>	completed	LLM directo	5.17 s
<code>overview_aapl</code>	completed	LLM directo	6.23 s
<code>returns_qqq_spy</code>	completed	LLM directo	9.87 s
<code>risk_qqq_spy</code>	completed	LLM directo	6.31 s
<code>technical_aapl</code>	completed	LLM directo	4.71 s

Tabla 8.3: Resultados del perfil universitario vLLM.

Los seis casos finalizaron sin fallback, sin warnings y sin recomendaciones directas de inversión. La prueba confirma que el sistema puede conectarse a un backend académico

propio sin cambiar el flujo principal.

8.5. Primera ejecucion comparativa completa

Ademas de las evaluaciones individuales por proveedor, se realizo una bateria comparativa con las cuatro variantes disponibles: determinista, Groq, Gemini y universidad vLLM. La ejecucion cubrio los seis ejemplos representativos en cada perfil, es decir, 24 ejecuciones en total. El resultado fue satisfactorio en los 24 casos: el modo determinista funciono como `sin_llm` y los tres perfiles externos funcionaron en modo `llm/directo`, sin fallback en esa bateria completa.

Variante	Rango aproximado por caso
Determinista	1.9 s – 3.4 s
Groq	3.5 s – 8.1 s
Gemini	6.4 s – 10.0 s
Universidad vLLM	6.7 s – 10.8 s

Tabla 8.4: Tiempos aproximados observados en la primera comparativa completa.

La lectura cualitativa de esta ejecucion es que las APIs no mejoran la precision numerica del sistema, ya que las metricas proceden del mismo pipeline determinista. Su aportacion se concentra en la comunicacion del resultado: **las respuestas son mas naturales, contextualizan mejor las metricas y se aproximan mas a un informe breve.** Este efecto fue especialmente visible en consultas como crecimiento historico, comparacion de retornos, riesgo historico y analisis tecnico, donde el modo determinista tiende a producir respuestas mas compactas.

8.6. Discusion de resultados

Los resultados apoyan la hipotesis de diseno del trabajo: una arquitectura hibrida permite combinar calculo financiero determinista con redaccion flexible asistida por LLM. El modo determinista actua como baseline y fallback, mientras que los proveedores externos mejoran la naturalidad de la explicacion final.

La comparativa tambien muestra que no todos los proveedores tienen el mismo comportamiento operativo. Groq resulto comodo para pruebas repetidas, Gemini apporto valor comparativo pero mostro limitaciones de cuota, y el endpoint universitario vLLM quedo validado como alternativa academica.

Capítulo 9

Aspectos eticos, seguridad y uso responsable

9.1. Delimitacion financiera del sistema

El sistema se orienta exclusivamente al analisis financiero historico y descriptivo. Sus respuestas deben interpretarse como explicaciones basadas en datos pasados, no como predicciones ni como recomendaciones de inversion. Esta delimitacion forma parte del diseno del MVP y debe mantenerse visible tanto en la memoria como en cualquier interfaz futura.

El prototipo calcula metricas como crecimiento, retornos, volatilidad, drawdown e indicadores tecnicos basicos. No realiza forecasting, no recomienda comprar o vender activos y no adapta decisiones a perfiles personales de riesgo.

9.2. Riesgos de interpretacion

Un riesgo relevante es que el usuario confunda una explicacion descriptiva con asesoramiento financiero. Para reducirlo, el sistema debe utilizar formulaciones prudentes, explicitar el periodo analizado y recordar que los resultados historicos no garantizan comportamientos futuros.

La capa LLM aumenta este riesgo si redacta respuestas demasiado persuasivas. Por ello, la configuracion actual solo permite al modelo interpretar resultados ya calculados y conserva mecanismos de fallback programatico.

9.3. Seguridad en la generacion y ejecucion de codigo

La generacion automatica de codigo introduce riesgos especificos. Un script mal generado podria fallar, calcular una metrica incorrecta o acceder a recursos no previs-

tos. En este proyecto, el riesgo se reduce mediante plantillas deterministas, entradas estructuradas, ejecucion controlada y captura explicita de salidas.

El codigo generado se ejecuta como proceso externo, con registro de salida estandar, salida de error y codigo de retorno. Esta estrategia permite revisar errores y evita que la respuesta final oculte fallos de ejecucion.

9.4. Privacidad y datos

Los datos utilizados son series historicas de mercado y no contienen datos personales. Aun asi, las claves de API de proveedores LLM se consideran informacion sensible y se almacenan exclusivamente en el fichero local `.env`, excluido del control de versiones.

En una version productiva, seria necesario revisar las condiciones de uso de los proveedores de datos y de modelos, asi como las politicas de retencion de informacion enviada a APIs externas.

9.5. Sostenibilidad, costes y recursos

El enfoque incremental reduce costes computacionales. El MVP no entrena modelos desde cero y puede ejecutarse en CPU sobre datos ya descargados. La capa LLM se activa solo cuando aporta valor, principalmente en interpretacion, y el modo determinista permite trabajar sin consumo de APIs externas.

Esta estrategia tambien mejora la sostenibilidad del desarrollo: evita ejecuciones innecesarias de modelos grandes, reduce dependencia de infraestructura GPU y permite comparar proveedores antes de elegir una configuracion de uso continuado.

9.6. Limitaciones de uso

El sistema no debe utilizarse como herramienta profesional de decision financiera sin validaciones adicionales. Su valor actual es academico y experimental: demostrar una arquitectura modular, trazable y robusta para automatizar analisis historicos.

Entre las limitaciones principales se encuentran el uso de un dataset acotado, la dependencia de datos de Yahoo Finance, la ausencia de validacion financiera profesional y la no incorporacion de informacion fundamental, noticias o variables macroeconomicas.

Capítulo 10

Conclusiones y trabajo futuro

10.1. Cumplimiento de objetivos

El trabajo desarrollado demuestra la viabilidad de un sistema multiagente para analisis financiero historico basado en una arquitectura modular, trazable y extensible. El MVP actual completa el flujo de extremo a extremo para seis intenciones analiticas, gestiona errores basicos de entrada y conserva una base determinista reproducible.

Los objetivos planteados se cumplen en una primera version funcional: se ha definido un workflow modular, se han implementado familias de agentes, se ha generado y ejecutado codigo Python de forma controlada, se han incorporado pruebas automatizadas y se ha preparado una integracion controlada con modelos de lenguaje.

10.2. Aportaciones principales

La primera aportacion del trabajo es la separacion clara entre calculo financiero e interpretacion textual. El sistema no delega las metricas en un LLM, sino que ejecuta codigo controlado y utiliza el modelo, cuando esta activo, para mejorar la explicacion final.

La segunda aportacion es la estrategia de fallback determinista. Gracias a ella, el MVP puede seguir funcionando aunque fallen las APIs externas, existan limites de cuota o se produzcan errores de red.

La tercera aportacion es la validacion comparativa de distintos proveedores LLM. Groq, Gemini y el modelo universitario vLLM se integran mediante una interfaz comun, lo que demuestra que la arquitectura no esta acoplada a un proveedor concreto.

10.3. Limitaciones actuales

El prototipo trabaja con un conjunto acotado de intenciones y con datos historicos ya descargados. No aborda prediccion futura, recomendacion de inversion, analisis fundamental ni incorporacion de noticias. Tampoco se ha activado todavia la generacion de codigo mediante LLM en la evaluacion principal, por considerarse una fase de mayor riesgo.

Otra limitacion es que la evaluacion se basa en casos representativos y pruebas automatizadas, pero aun no incluye una evaluacion extensa con usuarios ni una comparacion cuantitativa frente a plataformas financieras profesionales.

10.4. Lineas de trabajo futuro

Como trabajo futuro se plantea ampliar el catalogo de consultas, incorporar mas activos y fijar rangos temporales completamente reproducibles. Tambien seria interesante anadir visualizaciones al informe final, mejorar la capa de logging y extender las pruebas de robustez.

En la parte LLM, el siguiente paso razonable es mantener la interpretacion asistida y estudiar de forma controlada la activacion de la planificacion. La generacion de codigo mediante modelo deberia evaluarse solo despues de definir contratos estrictos, validadores automaticos y restricciones de seguridad adicionales.

Finalmente, una evolucion natural del proyecto seria construir una interfaz de usuario o API local que permita ejecutar el sistema de forma interactiva, manteniendo siempre visibles las limitaciones del analisis historico y la ausencia de recomendacion de inversion.

Referencias

- [1] LangChain, «LangGraph: Building Stateful, Multi-Actor Applications with LLMs,» 2024. dirección: <https://langchain-ai.github.io/langgraph/>
- [2] Groq, *Groq API Documentation*, 2026. dirección: <https://console.groq.com/docs>
- [3] Google AI for Developers, *Gemini API Documentation*, 2026. dirección: <https://ai.google.dev/gemini-api/docs>
- [4] R. Aroussi, *yfinance Python Library*, 2026. dirección: <https://github.com/ranaroussi/yfinance>

Apéndice A

Anexos

A.1. Ejecucion del MVP

El proyecto puede ejecutarse en modo determinista mediante los ejemplos incluidos en el repositorio:

```
python run_mvp.py --example growth_nvda
python run_mvp.py --example compare_nvda_amd
python run_mvp.py --example overview_aapl
python run_mvp.py --example returns_qqq_spy
python run_mvp.py --example risk_qqq_spy
python run_mvp.py --example technical_aapl
```

A.2. Configuracion de APIs

Las claves de API se guardan localmente en el fichero `.env`, que no debe subirse al repositorio. Para trabajar sin APIs externas:

```
LLM_ENABLED=false
```

Para activar Groq:

```
LLM_ENABLED=true
```

```
LLM_PROFILE=groq
```

Para activar Gemini:

```
LLM_ENABLED=true
```

```
LLM_PROFILE=gemini
```

Para activar el perfil universitario vLLM:

```
LLM_ENABLED=true  
LLM_PROFILE=university  
UNIVERSITY_BASE_URL=https://w1.etsisi.upm.es/vllm/v1  
UNIVERSITY_MODEL=openai/gpt-oss-20b
```

A.3. Documentacion interna de apoyo

Durante el desarrollo se conservaron notas tecnicas, informes de EDA y fichas de evaluacion en formato TXT. Estos documentos no sustituyen a la memoria, pero sirvieron como evidencias auxiliares para reconstruir decisiones de diseno y resultados experimentales. En particular, se utilizaron las notas de iteracion del MVP, el informe de EDA, las fichas de evaluacion determinista y LLM, la primera ejecucion comparativa completa y la guia de credenciales para reforzar las secciones de metodologia, datos, implementacion, integracion LLM y resultados.